Semantic Knowledge Representation and Reasoning: A First-Order Logic Approach for Explainable AI

Eashaan Thakur (2024114006)
Hemang Joshi (2024114015)

November 2025

1 Project Overview

This project develops a symbolic reasoning system that parses natural language into first-order logic (FOL) representations and performs automated reasoning to improve AI explainability and auditability. By combining rule-based parsing with logical inference, it addresses critical transparency gaps in modern AI systems—particularly in regulated domains like healthcare, finance, and law.

2 Research Question

Primary Question: How can semantic parsing improve explainability and auditability of AI systems by translating natural language into verifiable first-order logic representations?

Sub-questions:

1. How can rule-based parsing strategies ensure accurate and interpretable FOL translations?
2. How does symbolic reasoning enhance traceability and auditability compared to black-box models?
3. What are the main challenges in building efficient, real-time FOL parsers for explainable AI systems?

3 Motivation and Relevance

Modern AI models perform well but remain opaque ("black boxes"). Lack of interpretability poses risks in fairness, trust, and regulatory compliance. FOL provides:

- **Verifiability:** Formal validation by automated theorem provers.
- **Human Interpretability:** Experts can audit and modify logical rules directly.
- **Compositionality:** Complex reasoning decomposed into traceable inference steps.

A rule-based, symbolic system ensures complete transparency in how conclusions are derived, making AI decisions more reliable and explainable.

4 Literature Review

4.1 Key Works

- *Towards Trustable Explainable AI (IJCAI 2020)* – Establishes logic-based reasoning as essential for trustworthy AI.
- *Logic-Based Explainability: Past, Present & Future (arXiv 2024)* – Reviews formal and symbolic interpretability methods.
- *Better Metrics for Evaluating Explainable AI (AAMAS 2021)* – Defines fidelity, rule complexity, and stability as key explainability metrics.

4.2 Datasets and Benchmarks

- **FOLIO (Yale-LILY, 2022):** Logical reasoning dataset with annotated NL–FOL pairs.
- **LogicNLI (EMNLP 2021):** Benchmark for testing logical inference accuracy and robustness.

5 Methodology

5.1 System Architecture

Component 1: Natural Language to FOL Parser

- Converts text into FOL expressions with typed variables, quantifiers, and predicates.
- Implemented using rule-based templates and linguistic parsing (via NLTK or spaCy).

- Example: "All humans are mortal" $\rightarrow \forall x(\text{Human}(x) \rightarrow \text{Mortal}(x))$.

Component 2: FOL Reasoning Engine

- Performs automated inference using forward and backward chaining.
- Built using libraries such as `Kanren` or `pyDatalog`.
- Supports proof tracing for transparency of derived conclusions.

Component 3: Explainability Module

- Generates human-readable proof traces showing all inference steps.
- Visualizes logical dependencies and rule applications.
- Evaluates fidelity and completeness of explanations.

5.2 Baseline Systems

To measure performance relative to contemporary black-box systems, we compare the symbolic system to **LLM baselines** (zero-shot and few-shot prompting), using LLMs only as parsers or as end-to-end reasoners:

- **LLM-Parser baseline:** Prompt an LLM (e.g., GPT family) to translate NL $\rightarrow$ FOL. The resulting FOL is then fed to the same symbolic reasoner used for the rule-based system.
- **LLM-Reasoner baseline:** Ask an LLM to perform both parsing and reasoning, returning either an FOL translation plus proof-like text or direct conclusions with explanations.

All LLM experiments should be run with fixed random seeds where possible and repeated (e.g., 5 runs) to capture variance.

6 Evaluation Strategy

We evaluate on two levels: **parsing quality** (NL $\rightarrow$ FOL) and **reasoning / explainability quality** (inference correctness, proof traces, fidelity). For each metric below we state the computation and rationale.

6.1 Data splits and sample size

- Use FOLIO (or custom dataset) with the following recommended split: Train/dev/test where applicable. For evaluation against LLM baselines use a held-out **test set** of at least **200 examples** if available; otherwise use 50–100 carefully chosen, diverse examples.
- When dataset size is limited, use stratified sampling across logical constructs (quantifiers, connectives, nested structure).

6.2 Parsing metrics (NL → FOL)

1. Exact Match (EM). Fraction of test examples where the predicted FOL string exactly matches the gold FOL string after canonicalization (variable renaming and token normalization).

$$\text{EM} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{\text{canon}(\hat{f}_i) = \text{canon}(f_i)\}$$

where $\hat{f}_i$ is the predicted FOL, f_i the gold FOL, and $\text{canon}(\cdot)$ is canonicalization (normalize variable names, whitespace, predicate ordering where commutative).

2. Predicate-level Precision / Recall / F1. Treat each extracted predicate instance (predicate name + arity + arguments) as an item. Compute precision, recall, and F1:

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \qquad F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

where TP = correctly predicted predicates, etc.

3. BLEU (n-gram overlap). BLEU is applied to tokenized FOL strings (tokens = parentheses, predicate names, commas, quantifiers, variable names after canonicalization). Compute BLEU-1..4 with brevity penalty BP:

$$\text{BLEU} = BP \cdot \exp\Big(\sum_{n=1}^{4} w_n \log p_n\Big)$$

with $w_n = 1/4$ and p_n the clipped n-gram precision.
Note: BLEU measures surface similarity and can be misleading for logically equivalent but syntactically different formulas; therefore use BLEU alongside Exact Match and logical equivalence checks.

4. Tree Edit Distance (TED). If FOL is parsed into expression trees, compute normalized tree edit distance between predicted and gold trees:

$$\text{nTED} = 1 - \frac{\text{TED}(\hat{T}, T)}{\max(|\hat{T}|, |T|)}$$

report nTED where 1 = identical trees.

6.3 Logical equivalence and entailment checks

5. Logical Equivalence / Entailment Metrics. Use an automated theorem prover (e.g., Prover9, Z3, or SWI-Prolog) to evaluate semantic correctness beyond string matching:

- **Entailment Accuracy:** For each example with premises P and gold conclusion c, run the reasoner on P_{pred} (predicted FOL for premises) to check whether it entails c. Report fraction:

$$\text{EntailmentAcc} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\{P_{\text{pred},i} \models c_i\}$$

- **Bi-directional entailment / equivalence:** Check $P_{\text{pred}} \models c$ and $P_{\text{gold}} \models c$; also check whether $\hat{f}$ and f are logically equivalent (if computationally feasible) by checking mutual entailment.

6.4 Reasoning and explainability metrics

6. Reasoning Accuracy. Run the same reasoning queries on gold FOL and predicted FOL; reasoning accuracy is the fraction of queries where the result (entailment/no-entailment + same set of derived atoms) matches the gold-run.

7. Proof-Trace Overlap (Trace Jaccard). Let $\mathcal{T}_{\text{gold}}$ and $\mathcal{T}_{\text{pred}}$ be sets of rule applications or inference steps (represented as unique identifiers). Compute Jaccard index:

$$\text{TraceJaccard} = \frac{|\mathcal{T}_{\text{gold}} \cap \mathcal{T}_{\text{pred}}|}{|\mathcal{T}_{\text{gold}} \cup \mathcal{T}_{\text{pred}}|}$$

Higher values indicate more faithful explanations.

8. Fidelity (D). Fidelity measures how well the generated explanation reflects the actual inference process. Operationalize by comparing the explanation-derived proof

steps against the true inference log produced by the reasoner:

$$\text{Fidelity} = 1 - \frac{1}{K}\sum_{k=1}^{K} \mathbf{1}\{\text{step}_k^{\text{expl}} \notin \text{step\_log}^{\text{reasoner}}\}$$

where K is number of explanation steps.

9. Stability. Stability measures sensitivity of explanations to small perturbations in input (e.g., rewording). Compute variance of metrics (EM, F1, TraceJaccard) under a set of perturbations; lower variance = higher stability.

6.5 Efficiency and practical metrics

10. Runtime / Throughput. Measure average parsing time and inference time per query (milliseconds). Report median and 95% percentile to capture outliers.

11. Resource usage. Record memory usage and whether GPU is required (our rule-based system should be CPU-only).

6.6 Statistical analysis and reporting

- Run each baseline and our system on the same test examples. Report mean $\pm$ standard error for all metrics.
- Use paired statistical tests for metric comparisons:
  - Paired t-test when metric differences are approximately normal.
  - Wilcoxon signed-rank test for non-parametric comparisons.
- Report effect sizes (e.g., Cohen's d) and 95% confidence intervals (CI).
- Use bootstrap resampling (e.g., 1000 bootstrap samples) to estimate CIs for BLEU and F1 scores.

6.7 Procedure for LLM baseline experiments (reproducible)

1. **Prompt design:** Create a small set of prompts for zero-shot and few-shot translations (3–10 in-context examples).
2. **Deterministic setting:** Where available, run with fixed temperature or seed to reduce variance.
3. **Multiple runs:** Execute each prompt configuration 3–5 times and average results.

4. **Post-processing:** Canonicalize variable names and tokenization prior to metric computation.

5. **Reporting:** Report per-prompt and aggregate metrics; include example failure cases.

6.8 Caveats and best practices

- BLEU is informative for surface similarity but insufficient for semantic correctness; always pair with logical entailment and Exact Match.
- Canonicalization is crucial: variable renaming and predicate argument ordering (when commutative) must be normalized to avoid penalizing semantically equivalent outputs.
- When checking logical equivalence, be aware of undecidability in the general FOL case; restrict some checks to decidable fragments or use bounded-time theorem proving.

7 Expected Outcomes

1. Functional NL-to-FOL parser using rule-based templates.
2. Integrated reasoning engine capable of explainable inference.
3. Module for proof visualization and explanation generation.
4. Evaluation on a small subset of FOLIO or custom logical examples, comparing to LLM baselines using the defined metrics.

8 Success Criteria

- >70% parsing accuracy on validation examples (Exact Match / canonicalized).
- >65% reasoning accuracy on parsed statements.
- Proof traces available for all valid inferences.
- Demonstrable improvements in explanation fidelity and interpretability relative to LLM-parser baseline (statistically significant on chosen metrics).

9 Broader Impact and Future Work

Applications:

- Healthcare diagnostics with auditable reasoning chains.
- Legal and regulatory compliance checking.
- Transparent and fair financial decision-making systems.

Future Directions:

- Extend rule sets to handle complex natural language constructs.
- Integrate with domain-specific ontologies (medical, legal).
- Develop interactive interfaces for human-in-the-loop reasoning.

10 References

- IJCAI (2020) – Towards Trustable Explainable AI.
- arXiv (2024) – Logic-Based Explainability: Past, Present & Future.
- EMNLP (2021) – LogicNLI: Diagnosing FOL Reasoning Ability.
- Yale-LILY (2022) – FOLIO: Natural Language Reasoning with FOL.